Informe de Pruebas y Cobertura — SmartLogix

Generado: 2026-06-12T02:14:11.132Z

Resumen por servicio

Servicio	Statements	Branches	Functions	Lines	Coverage PDF
inventory-service	15.01%	35.2%	20.75%	16.37%	Abrir PDF
orders-service	0%	0%	0%	0%	Abrir PDF
payment-service	0%	0%	0%	0%	Abrir PDF
shipping-service	37.17%	29.41%	36.36%	40.62%	Abrir PDF
bff-service	8.75%	0%	4.65%	9.55%	Abrir PDF

Ejemplos de pruebas (fragmentos)

inventory-service

```
/**
 * Pruebas Unitarias — Inventory Service (Parcial 3)
 * Cobertura objetivo: ≥ 60%
 */
const ProductFactory = require('../factories/productFactory');
const { CircuitBreaker } = require('../middleware/circuitBreaker');

// ── Tests: ProductFactory ──────────────────────────────────────────
describe('ProductFactory', () => {
  test('crea producto físico con datos correctos', () => {
    const product = ProductFactory.create('physical', { name: 'Caja', price: '1500' });
    expect(product.name).toBe('Caja');
    expect(product.price).toBe(1500);
    expect(product.unit).toBe('unidad');
    expect(product.isActive).toBe(true);
  });

  test('crea producto digital con unidad licencia y minStock 0', () => {
    const product = ProductFactory.create('digital', { name: 'Software', price: '50000' });
    expect(product.unit).toBe('licencia');
    expect(product.minStock).toBe(0);
    expect(product.category).toBe('digital');
  });

  test('crea producto perecedero con minStock alto', () => {
    const product = ProductFactory.create('perishable', { name: 'Leche', price: '800' });
    expect(product.minStock).toBeGreaterThanOrEqual(10);
  });

  test('lanza error para tipo desconocido', () => {
```

orders-service

```
/**
 * Pruebas Unitarias — Orders Service
 */
```

```
// Simular el modelo Order
const generateOrderNumber = () => {
  const date = new Date();
  const num = Math.floor(Math.random() * 9000) + 1000;
  return `ORD-${date.getFullYear()}${(date.getMonth()+1).toString().padStart(2,'0')}-${num}`;
};

const calcTotal = (items) => items.reduce((s, i) => s + i.quantity * i.unitPrice, 0);

describe('Order Utilities', () => {
  test('genera número de orden con formato correcto', () => {
    const num = generateOrderNumber();
    expect(num).toMatch(/^ORD-\d{6}-\d{4}$/);
  });

  test('calcula total correctamente', () => {
    const items = [
      { quantity: 2, unitPrice: 1000 },
      { quantity: 1, unitPrice: 500 },
    ];
    expect(calcTotal(items)).toBe(2500);
  });

  test('calcula total con ítem de cantidad 0', () => {
    const items = [{ quantity: 0, unitPrice: 999 }];
    expect(calcTotal(items)).toBe(0);
```

payment-service

```
/**
 * Pruebas Unitarias — Payment Service
 */

// Simular lógica de validación sin llamar APIs reales
const validatePaymentRequest = ({ amount, orderId }) => {
  const errors = [];
  if (!amount || amount <= 0) errors.push('amount debe ser mayor a 0');
  if (!orderId) errors.push('orderId es requerido');
  return errors;
};

const getProviderName = (env) => {
  if (env === 'mercadopago') return 'MercadoPago';
  if (env === 'stripe') return 'Stripe';
  return 'Stripe'; // default
};

const formatAmountCLP = (amount) => Math.round(amount); // CLP sin decimales

describe('Payment Validations', () => {
  test('valida correctamente una solicitud válida', () => {
    const errors = validatePaymentRequest({ amount: 5000, orderId: 'ORD-001' });
    expect(errors).toHaveLength(0);
  });

  test('rechaza monto negativo', () => {
    const errors = validatePaymentRequest({ amount: -100, orderId: 'ORD-001' });
    expect(errors).toContain('amount debe ser mayor a 0');
  });
```

shipping-service

```
const express = require('express');
const request = require('supertest');

// Mockamos el modelo Shipment
const mockShipment = {
  findAll: jest.fn().mockResolvedValue([]),
  findByPk: jest.fn().mockResolvedValue(null),
  findOne: jest.fn().mockResolvedValue(null),
```

```
  create: jest.fn().mockImplementation(async (p) => ({ id: '123', ...p })),
  update: jest.fn().mockResolvedValue([1, [{ id: '123', status: 'DELIVERED' }]]),
};

jest.mock('../models', () => ({ Shipment: mockShipment }));

const shippingRouter = require('../routes/shipping');

describe('Shipping routes (unit)', () => {
  let app;
  beforeAll(() => {
    app = express();
    app.use(express.json());
    app.use('/shipping', shippingRouter);
  });

  test('GET /shipping returns 200 and array', async () => {
    mockShipment.findAll.mockResolvedValueOnce([]);
    const res = await request(app).get('/shipping');
    expect(res.status).toBe(200);
    expect(res.body).toHaveProperty('success', true);
    expect(Array.isArray(res.body.data)).toBe(true);
```

bff-service

```
const express = require('express');
const request = require('supertest');

jest.mock('../httpClient', () => ({
  get: jest.fn(),
  post: jest.fn(),
  put: jest.fn(),
  del: jest.fn(),
}));

const http = require('../httpClient');
const inventoryBff = require('../routes/inventoryBff');

function createApp() {
  const app = express();
  app.use(express.json());
  app.use('/', inventoryBff);
  app.use((err, req, res, next) => res.status(err.status || 500).json({ error: err.message ||
  return app;
}

describe('BFF inventory routes (unit)', () => {
  test('GET /products forwards response body', async () => {
    http.get.mockResolvedValue({ data: { success: true, data: [{ id: 1 }] } });
    const app = createApp();
    const res = await request(app).get('/products');
    expect(res.status).toBe(200);
    expect(res.body.success).toBe(true);
    expect(Array.isArray(res.body.data)).toBe(true);
  });
```

Comandos para reproducir pruebas

```
cd <service-folder>
npm install
npm test -- --coverage
```

Archivos generados

- reports/test-report.html
- reports/test-report.pdf

- reports/coverage/*.pdf (por servicio)